# Exercise: Managing a Student List Using OOP and ADT

Data Structures and Algorithms in Python

**Learning focus**

This exercise guides students from a real problem to object-oriented modeling, ADT design, abstract classes, inheritance, and concrete implementation using Python data structures. The goal is not to start with a complete solution, but to learn how to ask the right design questions before implementing code.

## 1. Problem description

Build a data structure to manage the students in a class.

Each student has the following information:

- student ID;
- full name;
- grade point average (GPA).

The data structure should support the following operations:

- add a student;
- search for a student by ID;
- search for students by full name;
- remove a student by ID;
- return the student with the highest GPA;
- optional: return the top five students with the highest GPA. If there are fewer than five students, return all students.

## 2. Identifying the object to be modeled

Answer the following questions before writing any code.

**Q1.** What is the basic real-world entity that the program needs to manage?

**Q2.** What pieces of data should be stored for one such entity?

**Q3.** If we use object-oriented programming, can these related data items be grouped into one object?

**Q4.** What would be a suitable class name for this object? Explain why.

**Q5.** What parameters should the constructor receive?

**Q6.** Should the ID, name, and GPA be stored as local variables inside `__init__`, or as attributes of the object? Why?

**Student task**

Complete the class skeleton below.

```
class __________:
    def __init__(self, __________):
        self.__________ = __________
        self.__________ = __________
        self.__________ = __________
```

Then create two different student objects using the following data:

```
SV001, Nguyen Van An, 8.2
SV002, Tran Thi Binh, 9.1
```

After creating the objects, answer:

a) Do the two objects belong to the same class?

b) Do they contain the same data?

c) Do they have the same identity?

## 3. From individual students to a student collection ADT

Assume that you now have many student objects.

**Q4.** Is one student object enough to manage the whole class? If not, what additional object or structure is needed?

**Q5.** What operations should this new structure support? List them from the problem description.

**Q6.** At this stage, do we need to decide whether students are stored in a `list`, a `dict`, or another structure?

**Q7.** Is it more important first to define *what operations* the structure provides?

**Q8.** If this abstract structure is called `StudentCollection`, what does it describe: the concrete storage method, or the operations available to users?

**Q9.** Explain in your own words why `StudentCollection` can be viewed as an Abstract Data Type (ADT).

**Student task**

Write a short ADT specification for `StudentCollection`. Do not write Python code yet. Describe the data and the operations in plain English or pseudocode.

## 4. Representing the ADT using an abstract class

In Python, an abstract class can be used to describe the methods that concrete implementations must provide.

**Q10.** Which Python module supports abstract base classes?

**Q11.** What is the role of `ABC`?

**Q12.** What is the role of `@abstractmethod`?

**Q13.** Why can the method bodies contain only `pass` at this stage?

**Q14.** What has not yet been decided when we write this abstract class?

**Student task**

Complete the abstract class below by adding all required operations from the problem description. Use `pass` in every method body.

```
from abc import ABC, abstractmethod


class StudentCollection(ABC):

    @abstractmethod
    def add(self, student):
        pass

    # Add the remaining required operations here.
```

## 5. From ADT to a concrete implementation

Now choose one concrete way to store students.

**Q15.** What is the simplest built-in Python data structure that can store many student objects?

**Q16.** If we choose a Python `list`, how should the storage attribute be initialized?

**Q17.** Should the concrete class be independent from `StudentCollection`, or should it inherit from `StudentCollection`? Why?

**Q18.** What is the difference between `StudentCollection` and `ListStudentCollection`?

**Q19.** Which class describes the interface? Which class describes how the data are stored?

**Student task**

Complete the class skeleton below. At this stage, you may still use `pass` in the method bodies.

```python
class ListStudentCollection(StudentCollection):

    def __init__(self):
        self.students = __________

    def add(self, student):
        pass

    # Complete all remaining methods required by the ADT.
```

The intended design relationship is:

`StudentCollection` → `ListStudentCollection` → Python `list`

# 6. Implementing the operations

## 6.1. Adding a student

**Q20.** If students are stored in `self.students = []`, which Python list method can add a new element?

**Q21.** Use that method to complete `add(self, student)`.

## 6.2. Searching by ID

**Q22.** What is the type of each element inside `self.students`?

**Q23.** When looping over `self.students`, which student attribute should be compared with the target ID?

**Q24.** If there are $n$ students, how many student objects may need to be checked in the worst case?

**Q25.** What does this imply about searching by ID using a list?

```python
def find_by_id(self, student_id):
    for student in self.students:
        # What condition should be checked here?
        pass
```

## 6.3. Searching by full name

**Q26.** Can two students have the same full name?

**Q27.** Should `find_by_name()` return one student object or a list of student objects?

**Q28.** How is this operation similar to and different from `find_by_id()`?

### 6.4. Removing by ID

**Q29.** Before removing a student by ID, what must be found first?

**Q30.** Can the logic of `find_by_id()` help with this operation?

**Q31.** When removing from a list, should you remove by object, by index, or by rebuilding a filtered list? Compare the options briefly.

### 6.5. Finding the student with the highest GPA

**Q32.** While scanning the list, what information should be remembered as the current best result?

**Q33.** Is it necessary to sort the whole list just to find one student with the highest GPA?

**Q34.** What should the method return if the collection is empty?

## 7. Optional extension: top five students

**Q35.** If we want the five students with the highest GPA, what simple strategy can be used?

**Q36.** Should the method be named `top_5_students()` or more generally `top_students(k)`? Which design is more flexible?

**Q37.** If the collection has only three students and the user requests the top five, how many students should be returned?

**Q38.** Should the original order of `self.students` be changed, or should the method return a sorted copy? Why?

**Student task**

Implement the optional method below.

```
def top_students(self, k=5):
    # Return at most k students with the highest GPA.
    pass
```

## 8. Thinking about another implementation

After completing `ListStudentCollection`, consider whether another implementation may be better for some operations.

**Q39.** If `find_by_id()` is called very frequently, is a list always the best choice?

**Q40.** Can a Python `dict` be used with student ID as the key and the `Student` object as the value?

**Q41.** If we create a dictionary-based implementation, should it provide the same operations as the list-based implementation?

**Q42.** What would be the same between the list-based and dictionary-based implementations?

**Q43.** What would be different between them?

```
class DictStudentCollection(StudentCollection):

    def __init__(self):
        self.students = {}

    # Implement the same ADT operations using a dictionary.
```

## 9. Final reflection

**Q44.** Identify the role of each component below:

- `Student`
- `StudentCollection`
- `ListStudentCollection`
- Python `list`

**Q45.** Complete the design chain below:

Real-world requirement
↓
Identify objects and data
↓
Design `Student`
↓
Identify required operations
↓
Design the ADT
↓
Abstract class / interface
↓
Concrete implementation
↓
Python data structure

**Q46.** Complete the two statements:

- OOP helps us organize related _____ and _____ inside objects.
- An ADT describes _____ a data type supports, without immediately deciding _____ those operations are implemented.

## 10. Expected learning outcome

By the end of this exercise, students should be able to explain the following chain of reasoning:

Problem → Object → Class → Operations → ADT
→ Abstract class → Inheritance → Implementation → Python data structure

The main lesson is that OOP helps organize data and behavior, while data structures and algorithms determine how operations are implemented efficiently.